DetectPoints 利用者の手引

1. はじめに

DetectPointsは、ハイキングマップなどの地図画像から円形のポイントマーカーを自動検出し、ポイントIDと座標を抽出するコンピュータビジョンツールです。地図上の赤色系（臙脂・赤）や白抜きの円形マーカーを認識し、近くにあるIDラベル（F4、F13、J11等）を自動で読み取ります。

2. システム要件

2.1 動作環境

- **OS**: Windows 10/11, macOS, Linux
- **Python**: 3.7以上
- **メモリ**: 最低4GB（高解像度画像処理時は8GB推奨）
- **ストレージ**: 1GB以上の空き容量

2.2 対応画像とポイント

- **形式**: PNG画像のみ
- **サイズ**: 制限なし（大きいほど処理時間が長くなります）
- **対象ポイント**: 円形の赤色系（臙脂・赤）または白抜きマーカー
- **IDラベル**: アルファベット1-2文字+数字の形式（F4、J11、SG等）
- **検出数**: 画像あたり10-200ポイント（通常は15前後）

3. インストール手順

3.1 Pythonの確認

コマンドプロンプト（Windows）またはターミナル（Mac/Linux）で以下を実行：

```
python --version
```

Python 3.7以上が表示されれば準備完了です。インストールされていない場合は、Python公式サイトからダウンロードしてください。

3.2 必要なライブラリのインストール

DetectPointsフォルダで以下のコマンドを実行：

```
pip install -r requirements.txt
```

以下のライブラリがインストールされます：

- opencv-python (>= 4.8.0)

- numpy (>= 1.24.0)
- pytesseract (>= 0.3.10)
- Pillow (>= 10.0.0)

3.3 Tesseract OCRのインストール（必須）

Windows

1. Tesseract-OCRからダウンロード
2. インストール時に「English」言語パックを選択（IDラベル認識用）
3. 環境変数にパスを追加（通常は自動設定されます）

Mac

```
brew install tesseract
```

Linux (Ubuntu/Debian)

```
sudo apt-get install tesseract-ocr tesseract-ocr-eng
```

3.4 インストール確認

正常にインストールできたか確認：

```
python detect_points.py samples/map-sample01.png
```

「検出されたポイント数: 15」と表示されれば成功です。

4. 基本的な使用方法

4.1 ポイント検出の実行

最も簡単な使用方法

```
python detect_points.py your_map.png
```

結果は入力ファイルと同じフォルダに `your_map_points.json` として保存されます。

出力ファイル名を指定する場合

```
python detect_points.py your_map.png -o custom_output.json
```

4.2 実行時の表示内容

処理中に以下の情報が表示されます：

```
円形ポイントを検出中...
検出された円: 20個
色によるフィルタリング中...
フィルタリング後: 15個
ポイント 1/15 のテキスト抽出中...
...
結果を保存しました: your_map_points.json

=== 検出完了 ===
検出されたポイント数: 15
出力ファイル: your_map_points.json
```

4.3 検出結果の確認

基本的な確認

```
python verify_points.py your_map.png your_map_points.json
```

インタラクティブモードでの確認（推奨）

```
python verify_points.py your_map.png your_map_points.json --interactive
```

画面に検出結果が表示され、以下のキー操作が可能：

- **'l' キー**: ポイント番号の表示/非表示
- **'i' キー**: ポイントIDの表示/非表示
- **'s' キー**: 現在の表示を画像保存
- **'r' キー**: 詳細レポート表示
- **'q' キー・ESC**: 終了

5. 出力データの理解

5.1 JSONファイルの構造

検出が完了すると、以下のようなJSONファイルが生成されます：

```
{
  "image_info": {
    "filename": "map-sample01.png",
```

```
      "width": 726,
      "height": 624
    },
    "points": [
      {
        "id": "F4",
        "name": "F4",
        "x": 512,
        "y": 126,
        "relative_x": 0.7052,
        "relative_y": 0.2019,
        "radius": 17
      },
      {
        "id": "J11",
        "name": "J11",
        "x": 348,
        "y": 176,
        "relative_x": 0.4793,
        "relative_y": 0.2821,
        "radius": 12
      }
    ],
    "total_points": 15
  }
```

5.2 データ項目の説明

- **id/name**: 検出されたポイントID（F4、J11等、OCRで抽出）
- **x, y**: 画像内の絶対座標（ピクセル単位、左上が原点）
- **relative_x, relative_y**: 相対座標（0.0-1.0の範囲、4桁精度）
- **radius**: 検出された円の半径（ピクセル単位）

5.3 ID認識について

- **成功例**: F4、F13、J11、SG、OX、ER、CS、58、20等
- **失敗時**: point_x_y形式（例：point_512_126）
- **対応形式**: アルファベット1-2文字 + 数字

6. 実践例とワークフロー

6.1 基本的なワークフロー

1. **画像準備**: ハイキングマップのPNG画像を用意
2. **検出実行**: `python detect_points.py hiking_map.png`
3. **結果確認**: `python verify_points.py hiking_map.png hiking_map_points.json --interactive`
4. **データ活用**: 生成されたJSONデータを他のアプリケーションで利用

6.2 サンプルデータでのテスト

付属のサンプルで動作確認：

```
# 検出実行
python detect_points.py samples/map-sample01.png

# 結果確認（インタラクティブ）
python verify_points.py samples/map-sample01.png samples/map-sample01_points.json
--interactive
```

期待結果：15個のポイント検出、複数のID認識

6.3 複数画像の一括処理

Windowsバッチ処理例

```
for %%f in (*.png) do (
    python detect_points.py "%%f"
    echo 処理完了: %%f
)
```

Linux/Macシェル処理例

```
for file in *.png; do
    python detect_points.py "$file"
    echo "処理完了: $file"
done
```

7. トラブルシューティング

7.1 検出に関する問題

「検出されたポイントが少ない」場合

原因と対処法：

- ポイントの色が薄い → コントラストを調整した画像を使用
- ポイントが小さすぎる → 高解像度画像を使用
- ポイントが円形でない → より円に近い形状のポイントを使用

「検出されたポイントが多すぎる」場合

原因と対処法：

- 背景ノイズ → 画像をクリーンアップ
- 他の円形要素 → 対象ポイント以外の円形要素を除去

- 画像解像度 → 適切なサイズに調整

「IDが正しく認識されない」場合

原因と対処法：

- Tesseract未インストール → Tesseractの再インストール
- 文字が不鮮明 → より高解像度な画像を使用
- 対応外の文字形式 → アルファベット+数字の形式を使用

7.2 技術的な問題

エラーメッセージと対処法

エラーメッセージ	原因	対処法
`ModuleNotFoundError: No module named 'cv2'`	OpenCV未インストール	`pip install opencv-python`
`TesseractNotFoundError`	Tesseract未インストール	Tesseractのシステムインストール
`ValueError: 画像ファイルを読み込めません`	画像ファイルの問題	ファイルパス・形式・破損を確認
`FileNotFoundError`	ファイルが見つからない	ファイルパスを確認

パフォーマンス問題

処理が遅い場合：

- 画像サイズを1000-2000ピクセル程度に調整
- 不要な背景要素を除去
- メモリ容量を増やす

8. 高度な使用方法

8.1 検証機能の活用

詳細レポートの出力

```
python verify_points.py your_map.png your_map_points.json -r detailed_report.txt
```

カスタム検証画像の生成

```
python verify_points.py your_map.png your_map_points.json -o
custom_verification.png --show-ids
```

8.2 データの活用方法

JSONデータの読み込み例（Python）

```
import json

with open('your_map_points.json', 'r', encoding='utf-8') as f:
    data = json.load(f)

for point in data['points']:
    print(f"ID: {point['id']}, 座標: ({point['x']}, {point['y']})")
```

相対座標の活用

相対座標を使用すると、画像サイズに依存しない位置情報として活用できます：

```
# 別の解像度での絶対座標計算
new_width, new_height = 1920, 1080
for point in data['points']:
    new_x = point['relative_x'] * new_width
    new_y = point['relative_y'] * new_height
    print(f"新座標: ({new_x:.0f}, {new_y:.0f})")
```

9. ベストプラクティス

9.1 画像準備のコツ

- **解像度**: 1000x1000ピクセル以上推奨
- **ファイル形式**: PNG形式のみ使用
- **コントラスト**: ポイントと背景を明確に区別
- **ポイント形状**: できるだけ円形に近い形状
- **ID表示**: ポイント近くに明確にID表示

9.2 効率的な作業フロー

1. **サンプルテスト**: samples/で動作確認
2. **小規模テスト**: 少数のポイントがある画像で動作確認
3. **パラメータ確認**: 検出結果を verify_points.py で確認
4. **本格処理**: 複数画像の一括処理
5. **品質チェック**: 生成されたJSONデータの妥当性確認

9.3 品質管理

- インタラクティブモードでの目視確認を必ず実施
- 期待される検出数と実際の検出数を比較

- ID認識率をサンプリング確認
- 座標精度を他の方法と照合

10. 技術的な詳細

10.1 検出アルゴリズムの特徴

- **複数パラメータ検出**: 2つの異なる設定で検出を実行
- **重複除去**: 距離ベースで重複する検出を自動除去
- **色フィルタリング**: HSV色空間での精密な色判定
- **OCR最適化**: 英数字のみに限定した高精度認識

10.2 精度向上のポイント

- ガウシアンブラーによるノイズ除去
- CLAHE（適応的ヒストグラム均等化）による前処理
- 正規表現パターンマッチングによるID抽出
- 複数の検出パラメータによる網羅性確保

11. よくある質問（FAQ）

Q1: どの程度の精度で検出できますか？

A1: 適切な画像（コントラスト良好、円形ポイント、明確なID表示）では、90%以上の検出率を実現できます。

Q2: 他の画像形式（JPEG等）には対応していますか？

A2: 現在はPNG形式のみに対応しています。他の形式は事前にPNGに変換してください。

Q3: 処理にはどのくらい時間がかかりますか？

A3: 1000x1000ピクセル程度の画像で15ポイント程度なら、通常1-2分程度です。

Q4: IDが認識されない場合はどうすればよいですか？

A4: 画像の解像度を上げるか、ポイント周辺の文字をより鮮明にしてください。

Q5: 商用利用は可能ですか？

A5: 使用している各オープンソースライブラリのライセンスに従ってください。

12. サポートとメンテナンス

12.1 ログの確認

処理中のメッセージは問題解決の重要な情報です。エラーが発生した場合は、表示されたメッセージを記録してください。

12.2 更新とバージョン管理

新しいバージョンがリリースされた場合は、requirements.txtを確認して依存関係を更新してください。

12.3 パフォーマンスモニタリング

大量の画像を処理する場合は、メモリ使用量やCPU使用率を監視し、システムリソースを適切に管理してください。

注意: このソフトウェアは研究・教育目的で開発されています。商用利用時は各コンポーネントのライセンスを確認してください。